

INDIAN INSTITUTE OF MANAGEMENT INDORE



Module 14: Capstone
Milind Naik | 20th September 2026





Capstone Response Playbook

AI-Augmented Software Delivery Lifecycle: BRD → PRD → FRD → HLD/LLD → Test Automation → Staged Approval → Deployment

A worked-example guide for Milestone 14: Executive AI Transformation Proposal – IIM Indore Executive Programme in Generative and Agentic AI for Business Applications

Purpose: this is not a model answer to copy verbatim. It is a worked example, grounded in one coherent use case, that shows how a strong response is built for each required section of the final proposal – so any student, regardless of technical background, can adapt it to their own organisation and still produce an executive-ready submission.



How to Use This Playbook

Every required section below is written in two layers.

CORE PATH

A complete, adaptable narrative that alone satisfies the module-level rubric (all 8 required sections, 25 points). Any student in the cohort, technical or not, can lift this and substitute their own company's facts.

STRETCH LAYER

The additional depth (the three-initiatives comparison, the HLD/LLD design branch, named prompting techniques, the private-endpoint architecture detail, the staged responsibility matrix) that is what separates a passing submission from one competing for the top band on the full 230-point cumulative rubric.

Both layers are given in full in this playbook – not because every student must write both, but because this cohort is being trained as future leaders, and the stretch layer is exactly the kind of thinking they should be exposed to even if their own submission only uses part of it.



Part 1 – The Capstone Requirement (Recap)

Milestone 14: Executive AI Transformation Proposal consolidates all 13 prior milestones into a single executive-ready report, submitted as a PDF, graded out of 230 points using the full cumulative rubric. The module-level assignment itself is graded on 8 required sections (25 points at module level; each also appears in the final 8 lines of the 230-point cumulative rubric).

The 8 Required Sections

01

Problem Overview

The business problem and why it matters to the organisation.

03

Generative AI Opportunity

What AI does, what stays human, how the workflow changes.

05

Financial Impact

Time saved, cost reduced, implementation cost, ROI.

07

Governance & Risk Management

Data privacy, hallucination mitigation, bias monitoring, audit trail, incident response.

02

Workflow Baseline

The current process: steps, manual work, bottlenecks.

04

Agent Design

The AI architecture: objective, planning, tools, memory, human-in-the-loop.

06

Strategy Alignment

How the initiative serves broader organisational strategy.

08

Implementation Roadmap

Phase 1 Pilot, Phase 2 Scaled Deployment, Phase 3 Autonomous Expansion.



What the Cumulative Rubric Actually Rewards

Because the final PDF is graded against every milestone's criteria rolled into one 230-point rubric, the strongest submissions treat each of the 8 sections above as a place to demonstrate specific, previously-taught techniques – not just to narrate in general terms. The table below is the cross-reference this playbook is built around.

Required Section	What the Deeper Rubric Is Actually Checking For
Problem Overview	Business context, workflow importance, initial value hypothesis
Workflow Baseline	Sequenced steps, manual vs. automated split, named bottlenecks, baseline metrics
Generative AI Opportunity	Named advanced prompting techniques (CoT, ToT, ReAct, self-consistency), guardrails, structured output
Agent Design	Agent objective, planning workflow, tool integrations, memory design, autonomy level, human oversight, responsibility matrix
Financial Impact	Worked time-savings and cost-per-hour calculations, implementation cost, 3-year ROI
Strategy Alignment	Three AI initiatives compared and one prioritised; named industry benchmarks
Governance & Risk Management	Named governance controls (privacy, hallucination mitigation, bias, audit, incident response), named risks
Implementation Roadmap	Three distinct, named phases with escalating scope



Part 2 – The Use Case at a Glance

This playbook uses one company throughout, so every section builds on the same facts instead of reading as eight disconnected essays. Substitute your own organisation's names and numbers; keep the structure.

Company

Meridian Software – a mid-size B2B SaaS product company (placeholder name; replace with your own organisation).

Function Affected

Product Management, Business Analysis, Engineering, QA, and Release Management – the full requirements-to-deployment chain for every product release.

Release Cadence

Monthly releases across three product lines; each release begins with a business stakeholder request and ends with production deployment.

The Workflow (Core)

Business stakeholder raises a need → Product Manager drafts a Business Requirement Document (BRD) → Product Manager expands it into a Product Requirement Document (PRD) → Business Analyst translates the PRD into a Functional Requirement Document (FRD) with detailed acceptance criteria → from the FRD, two workstreams run in parallel: QA authors test cases, and the Architecture team drafts High-Level and Low-Level Design (HLD/LLD) → staged approvals at each gate → development → deployment, which is AI-assisted (release notes, pre-deploy checklist, test-result summary prepared by AI) but human-triggered.

Text-form flow for turning into a diagram:

Stakeholder → BRD → PM → PRD → BA → FRD → [branches to both: Architect → HLD → LLD, and QA → Test Cases] → staged approvals → Dev → AI-prepped release checklist → human-triggered Deploy → post-deploy monitoring feeds the next BRD cycle



The Three AI Initiatives

STRETCH LAYER – for students chasing the top band

The Strategy Alignment section of the cumulative rubric wants three AI initiatives compared, with one prioritised. These aren't invented for the rubric – they're the natural decomposition of the same workflow, so comparing them is honest, not padding.

Initiative	Scope	Why It's a Candidate on Its Own
1. Requirements Generation Agent	BRD → PRD → FRD	Lowest risk, fastest to pilot, immediate cycle-time win
2. Design Generation Agent	FRD → HLD → LLD	Highest technical-risk tolerance needed; biggest quality upside if done well
3. Test & Release Automation Agent	FRD/LLD → test cases → release-readiness checklist	Directly reduces defect leakage and manual QA backlog

Prioritisation: this proposal recommends the integrated, end-to-end version – all three working off a shared traceability chain – over any single agent in isolation, because the compounding value (a requirement that is unambiguous, traceable all the way to its test case and design artifact) is larger than the sum of the three run independently. Feasibility, ROI and risk for each are quantified in the Financial Impact and Governance sections below.



The Architecture Pattern

STRETCH LAYER – for students chasing the top band

Vendor-agnostic by design, so it transfers to whichever cloud a given student's company already standardises on.

Model layer (Partner)

A managed foundation-model service running behind a private network endpoint inside the company's own cloud tenant – e.g. Azure OpenAI Service behind a Private Endpoint, AWS Bedrock behind VPC endpoints, or Vertex AI behind VPC Service Controls. Requirements, design and test content never cross the public internet.

Orchestration layer (Build)

The agent pipeline itself – prompt templates, the evaluation harness, the traceability store – built and owned in-house, since this is the company's actual differentiated capability.

Grounding

Retrieval-Augmented Generation (RAG) over the company's own BRD/PRD/FRD/HLD/LLD template library and architecture standards, so outputs match house style without fine-tuning the model itself.

Peripheral tooling (Buy)

Confluence/Jira for documents, a test-management tool (e.g. Zephyr/TestRail-class), and the existing CI/CD pipeline for deployment – none of this is rebuilt.

Why Partner, not Build, for the model layer: operating foundation-model infrastructure (capacity planning, patching, scaling, quantisation) is context work for a software product company, not core work – every hour spent there is an hour not spent on the product roadmap that differentiates the company in its market. This reasoning is the answer to the Build vs. Buy vs. Partner rubric line, and it should be stated explicitly, not left implicit.



Part 3 – The Response Playbook

Section by section, in the exact order the assignment asks for. Each section opens with what the rubric is actually checking for, then the core narrative, then the stretch layer.



Problem

Baseline

GenAI

Agents

Impact



1. Problem Overview

Rubric is checking for: organisational context, why the problem matters operationally or financially, which teams are affected, an initial value hypothesis.

CORE PATH – every student should be able to write this

Meridian Software ships monthly releases across three product lines. Every release begins the same way: a business stakeholder describes a need in a BRD, which a Product Manager turns into a PRD, which a Business Analyst turns into an FRD detailed enough for engineering and QA to work from. Today every one of those translations is manual, and each one is an opportunity for meaning to drift – a requirement that was clear in the stakeholder's head becomes ambiguous by the time it reaches a developer three documents later.

This matters financially, not just operationally: ambiguous requirements surface downstream as production defects, which are far more expensive to fix than a requirement clarified up front, and as slipped release dates, which have real consequences when a release date is tied to an enterprise customer's contractual go-live commitment. It matters organisationally because it consumes the time of exactly the people – Product Managers, Business Analysts, QA leads, Architects – whose judgement is Meridian's scarcest and most valuable resource, on documentation mechanics rather than on the judgement calls only they can make.

Teams affected: Product Management, Business Analysis, Engineering, QA, Architecture, and Release Management – effectively the entire path from a customer need to a shipped feature.

- ❏ **STRETCH LAYER** – for students chasing the top band: Initial value hypothesis (to be firmed up quantitatively in Financial Impact): a 30 - 40% reduction in requirements-to-release-ready cycle time, and a measurable reduction in defects traced to requirement ambiguity, are realistic first-pass targets based on the proportion of the current cycle spent on manual drafting and re-work rather than on judgement and review.



2. Workflow Baseline

Rubric is checking for: current workflow steps in sequence, manual vs. automated task identification, workflow bottlenecks, baseline metrics.

CORE PATH – every student should be able to write this

The current, fully manual process:

Step	Owner	Tool Today	Manual / Automated
1. Raise need	Business stakeholder	Email / meeting	Manual
2. Draft BRD	Product Manager	Word / Confluence	Manual
3. Draft PRD	Product Manager	Confluence	Manual
4. Draft FRD	Business Analyst	Confluence	Manual
5. Draft HLD/LLD	Architect	Confluence / diagramming tool	Manual, in parallel with step 6
6. Author test cases	QA Engineer	Zephyr / TestRail	Manual, in parallel with step 5
7. Approvals	PO, BA lead, Architecture Review Board, QA lead	Email / meetings	Manual
8. Deploy	Release Manager	CI/CD pipeline	Manual trigger, manual release notes

Bottlenecks: a multi-week requirements-to-design cycle driven by sequential document handoffs; a QA authoring backlog that routinely lags the FRD by days; approval delays because sign-off happens over email and meetings rather than a tracked workflow; and – specific to running HLD/LLD and test-case authoring in parallel from the same FRD – inconsistency between the design and the test cases when the two workstreams interpret an ambiguous requirement differently.

❏ **STRETCH LAYER** – for students chasing the top band: Current-state workflow diagram (to draw): a left-to-right swimlane with Stakeholder, PM, BA, Architect, QA, Release Manager as lanes, showing the BRD→PRD→FRD sequence as a single chain, then a visible fork at FRD into two parallel lanes (HLD/LLD and Test Cases) that re-converge at the approval gate before Deploy. The fork-and-reconverge shape is the one detail most student diagrams miss, and it is exactly the detail that explains the inconsistency bottleneck above.



3. Generative AI Opportunity

Rubric is checking for: which tasks AI performs vs. remain human, how the workflow changes, and (deeper rubric) named advanced prompting techniques with guardrails and structured output.

CORE PATH – every student should be able to write this

AI drafts; humans decide. At every one of the five document stages (PRD, FRD, HLD, LLD, test cases), the AI produces a first draft grounded in the company's own template library and prior approved documents, and a human – the same role who owns that document today – reviews, edits, and signs off. Nothing ships without a named human decision. What changes is the starting point: instead of a blank page, every reviewer starts from a structured, traceable draft.

Concretely: AI decomposes an approved PRD into FRD sections with structured acceptance criteria; AI drafts HLD/LLD content grounded in Meridian's own architecture standards; AI generates functional/UAT test cases from the FRD and unit/integration test cases from the LLD, each one tagged back to the requirement it came from. The workflow after AI is the same sequence of gates – it becomes draft-and-refine instead of author-from-scratch, and staged approvals become review-and-approve instead of author-and-approve.



Generative AI Opportunity – Prompting Techniques

STRETCH LAYER – for students chasing the top band

Naming the prompting techniques, rather than only describing outcomes, is what separates a strong answer here from a generic one:

Decomposition

The PRD-to-FRD step is prompted as an explicit breakdown instruction ("decompose this PRD into functional sections, each with numbered acceptance criteria"), rather than one open-ended "write the FRD" prompt.

Few-shot chain-of-thought

The prompt includes 2 - 3 prior, human-approved FRDs as worked examples, so the model's reasoning follows the pattern Meridian's own reviewers already trust.

Self-consistency

For the highest-stakes document (the FRD, since every downstream artifact inherits its ambiguity), the pipeline generates three candidate drafts and uses a separate scoring prompt against a rubric (completeness, traceability to the PRD, absence of ambiguous language) to select or merge the strongest one before it reaches the human reviewer.

Guardrails and output control: every generated document is required to be structured (numbered sections, tagged requirement IDs) rather than free text, so a downstream agent – or a human – can programmatically verify that every FRD line traces back to a PRD line, and every test case traces back to an FRD line, before the document is allowed to reach an approval gate.



4. Agent Design

Rubric is checking for: agent objective, planning workflow, tool integrations, memory design, autonomy level, human oversight, responsibility matrix.

CORE PATH – every student should be able to write this

Four agents, one shared memory. A Requirements Agent (BRD→PRD→FRD), a Design Agent (FRD→HLD→LLD), a Test Agent (FRD/LLD→test cases), and a Release-Support Agent (prepares release notes, a pre-deploy checklist, and a plain-English summary of test results for the human Release Manager – it does not trigger deployment itself). Tools: Confluence/Jira for reading and writing documents, the test-management system's API, and read-only status calls to the CI/CD pipeline. Memory: a shared traceability store that every agent reads from and writes to, carrying the lineage BRD→PRD→FRD→HLD/LLD→test case forward, so any artifact at any stage can be traced back to the original business need. Human-in-the-loop: every agent stops at a named human gate before its output moves to the next stage – no agent output reaches a downstream agent or a customer-facing artifact without a human approval in between.

This is deliberately a hybrid model, not a fully autonomous one: traditional structured checks (does every FRD line trace to a PRD line?) run alongside GenAI drafting, with a human making every judgement call that the checks can't make for it. That combination – structured validation plus generative drafting plus a human decision-maker – is what keeps the system trustworthy enough for a human to actually sign off on its output.



Agent Design – Staged Responsibility Matrix

STRETCH LAYER – for students chasing the top band

Staged responsibility matrix – naming who approves what, and what happens when they don't, is worth more on the rubric than a single generic "a human reviews it" line:

Stage	AI Agent Output	Human Approver	Escalation Trigger
PRD	Drafted PRD	Product Owner	2 consecutive rejections → escalate to Head of Product
FRD	Drafted FRD, self-consistency-scored	Business Analyst lead	Traceability check fails → auto-return to Requirements Agent before reaching BA
HLD / LLD	Drafted design docs	Architecture Review Board	Architectural deviation flagged → escalate to Chief Architect
Test Cases	Generated functional + unit/integration cases	QA Lead	Coverage gap vs. FRD detected → auto-return to Test Agent
Release Readiness	Release notes + pre-deploy checklist (AI-prepped, human-triggered)	Release Manager	Any failed pre-deploy check → deployment blocked, escalate to Release Manager + on-call Architect

Override mechanism: at every gate, the human approver can edit the AI output directly, reject and regenerate with feedback, or override entirely and author manually – the agent never has write access to the approved document store without that human action.



5. Financial Impact

Rubric is checking for: time savings estimation, cost-per-hour and annual cost reduction, implementation cost estimation, 3-year ROI.

CORE PATH – every student should be able to write this

A worked example (illustrative figures – replace with your own organisation’s numbers):

Line Item	Assumption	Result
Releases per year	3 product lines × 12 releases	36 release cycles/year
Time saved per cycle (PRD+FRD+design+test authoring)	40% reduction on ~60 hours of PM/BA/Architect/QA effort per cycle	~24 hours saved/cycle
Annual hours saved	24 hours × 36 cycles	~864 hours/year
Blended loaded cost per hour (PM/BA/Architect/QA)	Illustrative	₹2,500/hour
Annual cost saving	864 hours × ₹2,500	~₹21.6 lakh/year
Implementation cost (Year 1)	Private-endpoint model usage + orchestration build + integration effort	~₹35 lakh (one-time + Year-1 run cost)
3-Year ROI	(3-year cumulative savings – 3-year cumulative cost) / 3-year cumulative cost	Positive from Year 2 onward once orchestration build cost is amortised

❏ **STRETCH LAYER** – for students chasing the top band: An honest ROI line names the trade-off it accepted, not just the upside: the Partner (managed, private-endpoint) architecture costs more per unit than a public API and takes longer to stand up than "just calling ChatGPT" – that incremental cost is the price of the governance requirement in Section 7, and it belongs in this table as a named line item ("governance premium"), not hidden inside a general implementation-cost number. A reviewer who sees that trade-off named explicitly reads the whole proposal as more credible.



6. Strategy Alignment

Rubric is checking for: alignment with strategic objectives, and (deeper rubric) three AI initiatives compared with one prioritised, plus named industry benchmarks.

CORE PATH – every student should be able to write this

Faster, more reliable releases are not a side effect for a software product company – they are the strategy. Reducing requirements-to-release cycle time shortens time-to-market against competitors; reducing defect leakage protects renewal rates and customer trust, which for a B2B SaaS company is the single biggest lever on lifetime value; and freeing PM/BA/QA capacity from documentation mechanics lets Meridian spend that capacity on the judgement calls that actually differentiate its product – the same core-versus-context argument that justified the Partner architecture decision, now applied to people instead of infrastructure.



Strategy Alignment – Three-Initiatives Comparison

STRETCH LAYER – for students chasing the top band

The three-initiatives comparison from Part 2, scored:

Initiative	Feasibility	ROI	Implementation Risk	Strategic Impact
1. Requirements Generation	High – narrowest scope, fastest pilot	Medium – cycle-time win only	Low	Medium
2. Design Generation	Medium – needs architecture standards codified first	High – biggest quality upside	Medium-High – design errors are expensive	High
3. Test & Release Automation	High	Medium-High – directly cuts defect leakage	Low-Medium	High
Integrated (all three)	Medium – more to build and govern	Highest – compounding, traceable value	Medium – mitigated by staged rollout (Section 8)	Highest

Prioritisation: the integrated pipeline is recommended over any single initiative because the traceability chain is where the real value compounds – a test case that can be proven to trace to an unambiguous FRD line is worth more than either the design agent or the test agent alone.

Industry benchmarks (name at least two, described factually, not as marketing copy): GitHub Copilot Workspace and Atlassian Intelligence both extend GenAI into requirements- and documentation-adjacent SDLC work inside existing developer tools; Qodo (formerly CodiumAI) is a named example of AI-generated test cases as a distinct product category. Citing category-level, factual comparators like these is stronger than inventing a fictional "Company 1 / Company 2" case study.



7. Governance & Risk Management

Rubric is checking for: data privacy protections, hallucination mitigation, bias monitoring, monitoring/audit trails, incident response.

CORE PATH – every student should be able to write this

Because Meridian's BRD/PRD/FRD/HLD/LLD content is its own competitive IP and architecture – not generic text – the governance model is built around never letting that content leave the company's control, not just around AI safety in the abstract:

Data privacy

The model runs behind a private network endpoint inside Meridian's own cloud tenant, so requirement, design and test content never crosses the public internet. Separately, and just as important, Meridian's agreement with the model provider includes a zero-data-retention / no-training clause – network isolation and a contractual no-retention commitment are two different controls, and both are named explicitly rather than assumed to be the same thing.

Access control

Every agent runs under its own scoped service account (RBAC), so the Design Agent cannot write to the Test Agent's outputs and no agent can bypass its human approval gate.

Audit trail

Every generated artifact and every human decision (approve, reject, edit, escalate) is logged, so any document in production can be traced back to who approved it and what the AI's original draft looked like.

Hallucination mitigation

The structured-output and traceability checks from Section 3, plus mandatory human sign-off at every gate – no agent output reaches a customer-facing artifact unreviewed.



Governance — Risk / Mitigation Matrix

STRETCH LAYER – for students chasing the top band

Risk / mitigation / required capability, named explicitly (a private, network-isolated model solves the leakage risk – it does not solve model-quality risk, and a strong governance section says so rather than implying private hosting makes everything safe):

Key Risk	Mitigation	Capability Required
Hallucinated or incomplete requirements/design content	Structured-output validation, self-consistency scoring, mandatory human sign-off	Prompt/evaluation engineering
Prompt injection via a malformed or malicious source BRD	Input sanitisation before the Requirements Agent, human review before anything downstream	Agent security review
Insider over-permissioning inside the private perimeter	RBAC-scoped service accounts, least-privilege per agent, audit logging	Cloud IAM discipline
Vendor dependency / lock-in on the model provider	Named and accepted explicitly as a strategic trade-off; orchestration layer built to be model-agnostic where practical	Vendor management

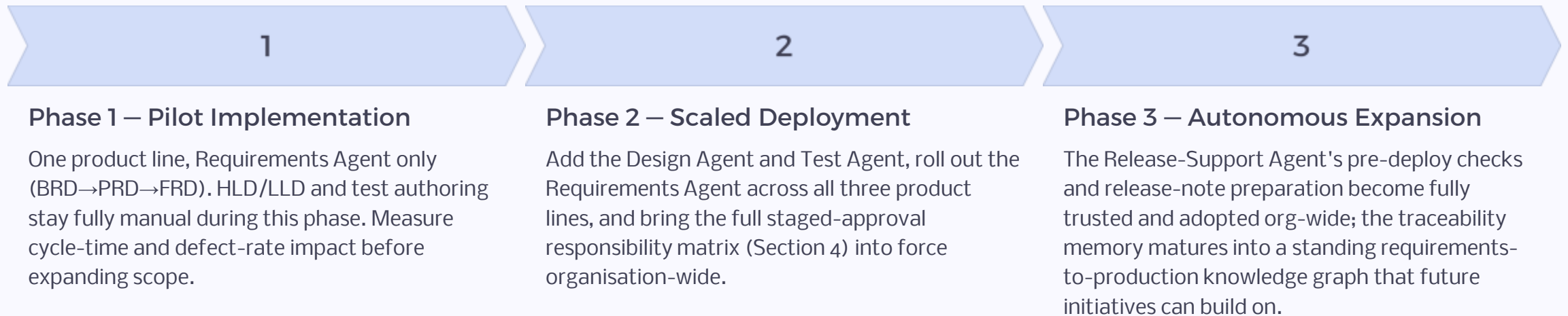
Incident response: a detected failure (e.g. a traceability check failing in production, or an approved document later found to contain an AI error) triggers a defined path – the Release Manager and on-call Architect are notified, the affected artifact is flagged in the traceability store, and root-cause review happens before the same agent stage is trusted again for that document type.



8. Implementation Roadmap

Rubric is checking for: Phase 1 Pilot, Phase 2 Scaled Deployment, Phase 3 Autonomous Expansion, each clearly described.

CORE PATH – every student should be able to write this



- ❑ **STRETCH LAYER** – for students chasing the top band: A note on scope, stated honestly rather than glossed over: this roadmap keeps deployment itself AI-assisted but human-triggered throughout all three phases – the Release-Support Agent prepares checks and notes, a person still decides to deploy. A fully autonomous deployment agent (AI-monitored, with automatic rollback on failure) is a legitimate further step some organisations may eventually want, but it is deliberately scoped out of this proposal: it requires a materially higher trust bar in the agent's judgement than Phases 1-3 assume, and naming it as a considered-and-declined option is more credible than either ignoring it or overreaching into it without the monitoring maturity to support it.



Appendix – Self-Check Before Submission

Before submitting, confirm the response (in whatever form a given student writes it) does all of the following. This list follows the assignment's own submission checklist, expanded with the stretch-layer items this playbook adds.

Core (required for the module-level 25 points)

- ☐ Problem overview names the affected teams and the business/financial stakes, not just "AI could help here".
- ☐ Workflow baseline shows the steps in sequence, with a manual-vs-automated split and at least one named bottleneck.
- ☐ GenAI opportunity names which tasks AI performs and which stay human, for every stage – not just one.
- ☐ Agent design names the agent(s), the tools they call, the memory/traceability mechanism, and at least one human approval gate.
- ☐ Financial impact includes an actual calculation – hours, cost per hour, and a savings or ROI figure – not just a qualitative claim.
- ☐ Strategy alignment ties the initiative to a named organisational goal, not a generic "innovation" statement.
- ☐ Governance section names at least one specific control for privacy, one for hallucination mitigation, and one for audit/monitoring.
- ☐ Roadmap has three distinct phases with escalating scope, not three restatements of the same thing.
- ☐ Name, Batch/Cohort, and Date of Submission appear at the start of the file, and the final export is a single PDF.



Self-Check — Stretch (Top Band)

Stretch (for the top band on the full 230-point cumulative rubric)

- ☐ At least two named advanced prompting techniques (e.g. decomposition, few-shot CoT, self-consistency) with a one-line explanation of why each was used.
- ☐ A responsibility matrix naming who approves each stage and what triggers escalation.
- ☐ Three AI initiatives compared on feasibility, ROI, risk and strategic impact, with one explicitly prioritised and justified.
- ☐ The Build vs. Buy vs. Partner decision is answered separately for the model layer, the orchestration layer, and peripheral tooling – not as one blanket answer.
- ☐ At least one named, factual industry benchmark (not an invented case study).
- ☐ A named, accepted trade-off somewhere in the proposal (cost, vendor lock-in, or scope declined) – a proposal that only lists upside reads as less credible than one that also owns a cost.
- ☐ Current-state and future-state workflow diagrams are attached (described in Part 2 of this playbook), not just narrated in text.





Get Ready to Rate!

Please select your rating from the options provided during the poll.

Rate today's session from 1 to 5, where 1 is the lowest and 5 being the Highest
Your input helps us improve our sessions.

Thank you for your feedback! 😊



Over to you!

